CS473 - Algorithms I

Lecture 10
Dynamic Programming

C. Aykanat and others
Computer Engineering Department, Bilkent University

Introduction

- An algorithm design paradigm like divide-and-conquer
- "Programming": A tabular method (not writing computer code)

  Older sense of planning or scheduling, typically by filling in a table
- Divide-and-Conquer (DAC): subproblems are independent
- Dynamic Programming (DP): subproblems are not independent
- Overlapping subproblems: subproblems share sub-subproblems
  - In solving problems with overlapping subproblems
    - A DAC algorithm does redundant work
      - Repeatedly solves common subproblems
    - A DP algorithm solves each problem just once
      - Saves its result in a table

C. Aykanat and others
Computer Engineering Department, Bilkent University

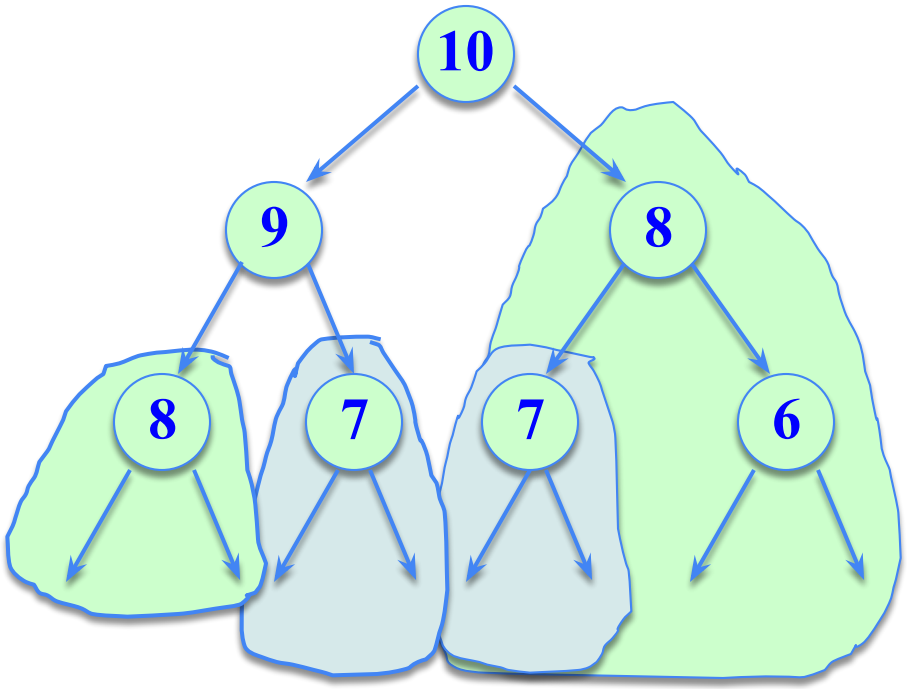
Example: Fibonacci Numbers (Recursive Solution)

Reminder:

$F(0) = 0$ and $F(1) = 1$

$F(n) = F(n-1) + F(n-2)$

```
REC-FIBO(n)
  if n < 2
      return n
  else
     return REC-FIBO(n-1)
            + REC-FIBO(n-2)
```



Overlapping subproblems in different recursive calls. Repeated work!

Example: Fibonacci Numbers (Recursive Solution)

Recurrence:

$$T(n) = T(n-1) + T(n-2) + 1$$

$\Rightarrow$ exponential runtime

Recursive algorithm inefficient because it recomputes the same F(i) repeatedly in different branches of the recursion tree.

Example: Fibonacci Numbers (Bottom-up Computation)

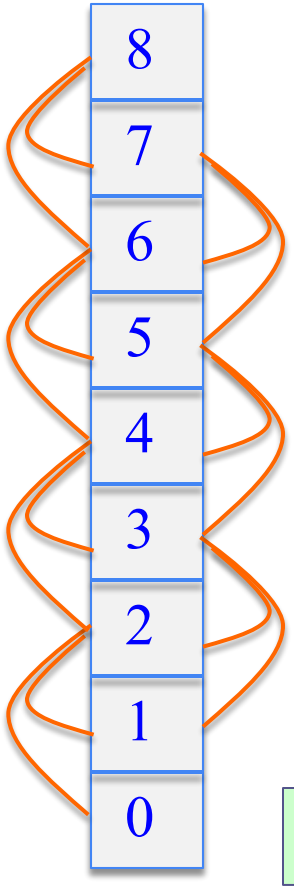
Reminder:

$F(0) = 0$ and $F(1) = 1$

$F(n) = F(n-1) + F(n-2)$

ITER-FIBO(n)

```
F[0] = 0
F[1] = 1
for i = 2 to n do
    F[i] = F[i-1] + F[i-2]
return F[n]
```



Runtime: $\Theta(n)$

C. Aykanat and others
Computer Engineering Department, Bilkent University

Optimization Problems

- DP typically applied to optimization problems
- In an optimization problem
  - There are many possible solutions (feasible solutions)
  - Each solution has a value
  - Want to find an optimal solution to the problem
    - A solution with the optimal value (min or max value)
  - Wrong to say "the" optimal solution to the problem
    - There may be several solutions with the same optimal value

Development of a DP Algorithm

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution in a bottom-up fashion
4. Construct an optimal solution from the information computed in Step 3

Example: Matrix-chain Multiplication

- *Input*: a sequence (chain) $\langle A_1, A_2, \ldots, A_n \rangle$ of n matrices
- *Aim*: compute the product $A_1 \cdot A_2 \cdot \ldots \cdot A_n$
- A product of matrices is fully parenthesized if
  - It is either a single matrix
  - Or, the product of two fully parenthesized matrix products surrounded by a pair of parentheses.

$$(A_i(A_{i+1}A_{i+2} \ldots A_j))$$

$$((A_iA_{i+1}A_{i+2} \ldots A_{j-1})A_j)$$

$$((A_iA_{i+1}A_{i+2} \ldots A_k)(A_{k+1}A_{k+2} \ldots A_j)) \quad \text{for } i \le k < j$$

  - *All parenthesizations yield the same product; matrix product is associative*

C. Aykanat and others
Computer Engineering Department, Bilkent University

Matrix-chain Multiplication: An Example Parenthesization

- *Input*: $\langle A_1, A_2, A_3, A_4 \rangle$
- 5 distinct ways of full parenthesization

  $(A_1(A_2(A_3A_4)))$
  $(A_1((A_2A_3)A_4))$
  $((A_1A_2)(A_3A_4))$
  $((A_1(A_2A_3))A_4)$
  $(((A_1A_2)A_3)A_4)$

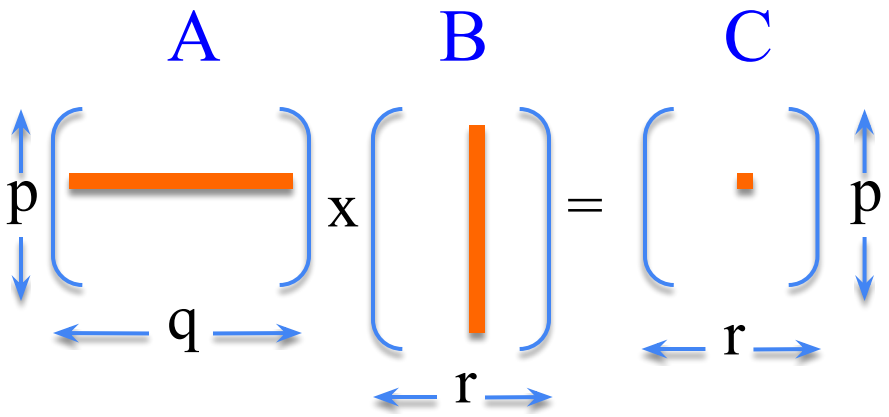
- The way we parenthesize a chain of matrices can have a dramatic effect on the cost of computing the product

C. Aykanat and others
Computer Engineering Department, Bilkent University

Reminder: Matrix Multiplication

MATRIX-MULTIPLY(A, B)

```
if cols[A] ≠ rows[B] then
error("incompatible dimensions")
for i ←1 to rows[A] do
    for j←1 to cols[B] do
        C[i,j] ← 0
        for k←1 to cols[A] do
            C[i,j]←C[i,j]+A[i,k]·B[k,j]
return C
```



rows(A) = p
cols(A) = q

rows(B) = q
cols(B) = r

rows(C) = p
cols(C) = r

Reminder: Matrix Multiplication

MATRIX-MULTIPLY(A, B)

```
if cols[A] ≠ rows[B] then
error("incompatible dimensions")
for i ←1 to rows[A] do
    for j←1 to cols[B] do
        C[i,j] ← 0
        for k←1 to cols[A] do
            C[i,j]←C[i,j]+A[i,k]·B[k,j]
return C
```

A: p x q
B: q x r

C: p x r

of mult-add ops
= rows[A] x cols[B] x cols[A]

of mult-add ops = p x q x r

C. Aykanat and others
Computer Engineering Department, Bilkent University

Matrix Chain Multiplication: Example

A_1: 10x100 A_2: 100x5 A_3: 5x50

Which paranthesization is better? $(A_1A_2)A_3$ or $A_1(A_2A_3)$?

$$10\left[\begin{matrix}100\\A_1\end{matrix}\right] \times 100\left[\begin{matrix}5\\A_2\end{matrix}\right] = 10\left[\begin{matrix}5\\A_1A_2\end{matrix}\right]$$

of ops: 10 . 100 . 5 = 5000

$$10\left[\begin{matrix}5\\A_1A_2\end{matrix}\right] \times 5\left[\begin{matrix}50\\A_3\end{matrix}\right] = 10\left[\begin{matrix}50\\A_1A_2A_3\end{matrix}\right]$$

of ops: 10 . 5 . 50 = 2500

Total # of ops: 7500

C. Aykanat and others
Computer Engineering Department, Bilkent University

Matrix Chain Multiplication: Example

A_1: 10x100 A_2: 100x5 A_3: 5x50

Which paranthesization is better? $(A_1A_2)A_3$ or $A_1(A_2A_3)$?

$$\underset{100}{\left[\begin{array}{c}5\\ A_2\end{array}\right]} \times \underset{5}{\left[\begin{array}{c}50\\ A_3\end{array}\right]} = \underset{100}{\left[\begin{array}{c}50\\ A_2A_3\end{array}\right]}$$

of ops: 100 . 5 . 50 = 25000

$$\underset{10}{\left[\begin{array}{c}100\\ A_1\end{array}\right]} \times \underset{100}{\left[\begin{array}{c}50\\ A_2A_3\end{array}\right]} = \underset{10}{\left[\begin{array}{c}50\\ A_1A_2A_3\end{array}\right]}$$

of ops: 10 . 100 . 50 = 50000

Total # of ops: 75000

C. Aykanat and others
Computer Engineering Department, Bilkent University

Matrix Chain Multiplication: Example

A_1: 10x100 A_2: 100x5 A_3: 5x50

Which parenthesization is better? $(A_1A_2)A_3$ or $A_1(A_2A_3)$?

In summary: $(A_1A_2)A_3$ ⇒ # of multiply-add ops: 7500

$A_1(A_2A_3)$ ⇒ # of multiple-add ops: 75000

⇒ First parenthesization yields 10x faster computation